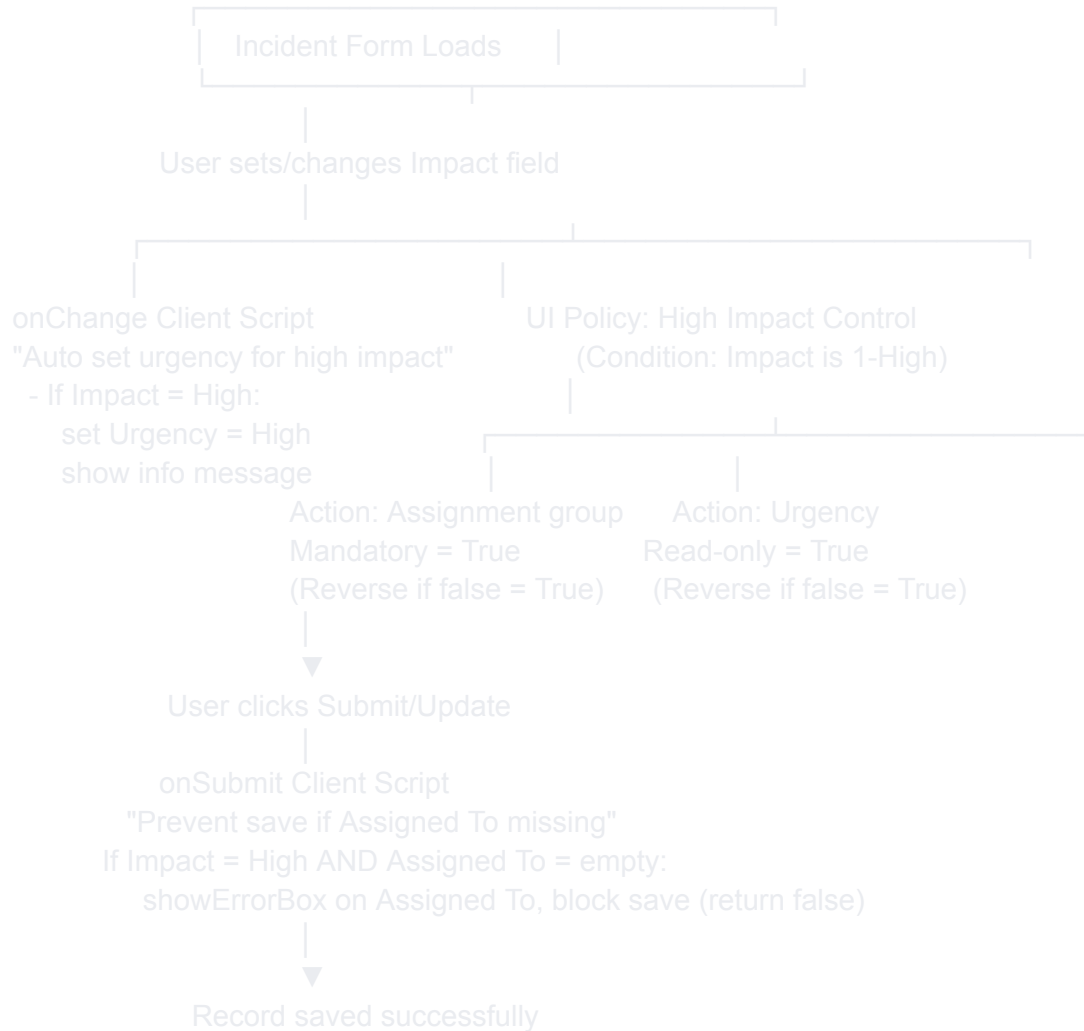


Solution — Detailed Design

Architecture Flow



Separately, on the Incident LIST view:

onCellEdit Client Script "Prevent state change via list edit"

- Any inline edit of State → alert + callback(false) → edit is rejected

Field Behavior Matrix

Field	Trigger	Behavior	Mechanism
Assignment group	Impact = High	Becomes mandatory	UI Policy Action
Assignment group	Impact ≠ High	Mandatory removed	UI Policy — Reverse if false
Urgency	Impact = High	Becomes read-only	UI Policy Action

Urgency	Impact $\neq$ High	Editable again	UI Policy — Reverse if false
Urgency	Impact changed to High	Auto-set to "1 - High" + info message	Client Script (onChange)
Assigned to	Impact = High, on Submit	Save blocked + error if empty	Client Script (onSubmit)
State	Edited from list view	Edit blocked with alert	Client Script (onCellEdit)
State	Edited from the form	Allowed, saves normally	No restriction on form

Component Design

UI Policy — High Impact Control: Table = Incident, Active = true, Condition = Impact is 1-High, Reverse if false = true, On load = true.

UI Policy Actions: Assignment group → Mandatory = True; Urgency → Read only = True.

Client Script — Auto set urgency for high impact: Table = Incident, Type = onChange, Field = Impact, Active = true.

Client Script — Prevent save if Assigned To missing: Table = Incident, Type = onSubmit, Active = true.

Client Script — Prevent state change via list edit: Table = Incident, Type = onCellEdit, Field = State, Active = true.

Design Rationale

UI Policy is used wherever the requirement is purely declarative (mandatory/read-only tied to a condition) — "Reverse if false" gives automatic two-way behavior for free. Client Scripts are used only where UI Policy can't express the logic: auto-populating a value with a message, custom save-time validation with a specific error, and intercepting list-level inline edits.